

```
from google.colab import files
uploaded = files.upload()
```

housing_ma...t_dataset.csv

housing_market_dataset.csv(text/csv) - 29929 bytes, last modified: 01/06/2026 - 100% done
Saving housing_market_dataset.csv to housing_market_dataset.csv

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load the uploaded CSV
df = pd.read_csv('housing_market_dataset.csv')

# Read first 5 rows
print("First 5 rows:")
print(df.head()) # This will show first 5 rows
```

First 5 rows:

	House_ID	Neighborhood	House_Type	SqFt_Living	Bedrooms	Bathrooms
0	1001	Rural	Single Family	3013.0	1.0	1.5
1	1002	Waterfront	Condo	2423.0	5.0	3.5
2	1003	Urban	Condo	1477.0	4.0	2.5
3	1004	Waterfront	Apartment	1985.0	2.0	1.5
4	1005	Waterfront	Single Family	NaN	2.0	1.5

	Year_Built	Distance_to_City_Center	Has_Pool	Condition_Score	Price
0	1952	5.42	0	3	723515
1	2019	13.20	1	5	711527
2	2000	4.71	0	5	546717
3	1968	26.76	0	2	478695
4	1991	10.70	1	2	469769

```
print("first 10 row:")
print (df.head())
```

first 10 row:

	House_ID	Neighborhood	House_Type	SqFt_Living	Bedrooms	Bathrooms
0	1001	Rural	Single Family	3013.0	1.0	1.5
1	1002	Waterfront	Condo	2423.0	5.0	3.5
2	1003	Urban	Condo	1477.0	4.0	2.5
3	1004	Waterfront	Apartment	1985.0	2.0	1.5
4	1005	Waterfront	Single Family	NaN	2.0	1.5

	Year_Built	Distance_to_City_Center	Has_Pool	Condition_Score	Price
0	1952	5.42	0	3	723515

1	2019	13.20	1	5	711527
2	2000	4.71	0	5	546717
3	1968	26.76	0	2	478695
4	1991	10.70	1	2	469769

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   House_ID                             500 non-null    int64
1   Neighborhood                         500 non-null    object
2   House_Type                           500 non-null    object
3   SqFt_Living                          485 non-null    float64
4   Bedrooms                             490 non-null    float64
5   Bathrooms                           500 non-null    float64
6   Year_Built                           500 non-null    int64
7   Distance_to_City_Center              500 non-null    float64
8   Has_Pool                             500 non-null    int64
9   Condition_Score                      500 non-null    int64
10  Price                                500 non-null    int64
dtypes: float64(4), int64(5), object(2)
memory usage: 43.1+ KB
```

```
df.describe()
```

	House_ID	SqFt_Living	Bedrooms	Bathrooms	Year_Built	Distance_
count	500.000000	485.000000	490.000000	500.0000	500.000000	
mean	1250.500000	2123.336082	3.004082	2.5420	1987.052000	
std	144.481833	588.621158	1.415653	0.9865	20.889088	
min	1001.000000	481.000000	1.000000	1.0000	1950.000000	
25%	1125.750000	1753.000000	2.000000	1.5000	1970.000000	
50%	1250.500000	2127.000000	3.000000	2.5000	1988.000000	
75%	1375.250000	2495.000000	4.000000	3.5000	2004.250000	
max	1500.000000	3679.000000	5.000000	4.0000	2023.000000	

```
#Feature engineering
df['Price_perSqFt'] = df['Price'] / df['SqFt_Living']
print(df[['Price_perSqFt']])

# Status or Premium based on 75th percentile
```

```
df['Status or Premium'] = np.where(df['Price'] > df['Price'].quantile(0.75), 'Premium', 'Standard')
print(df['Status or Premium'])
```

```

      Price_perSqFt
0      240.131099
1      293.655386
2      370.153690
3      241.156171
4           NaN
..          ...
495     276.349716
496     254.297978
497     335.086957
498     265.794977
499     266.815976

```

```
[500 rows x 1 columns]
```

```

0      premium
1      premium
2      standard
3      standard
4      standard
..          ...
495     standard
496     standard
497     standard
498     standard
499     standard

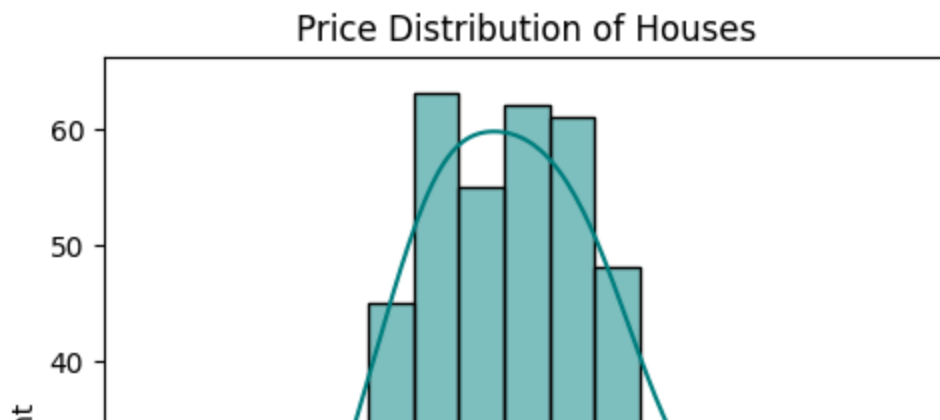
```

```
Name: Status or Premium, Length: 500, dtype: object
```

```

# Cell 7: Price Distribution plot
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
sns.histplot(df['Price'], kde=True, color='teal')
plt.title('Price Distribution of Houses')
plt.show()

```

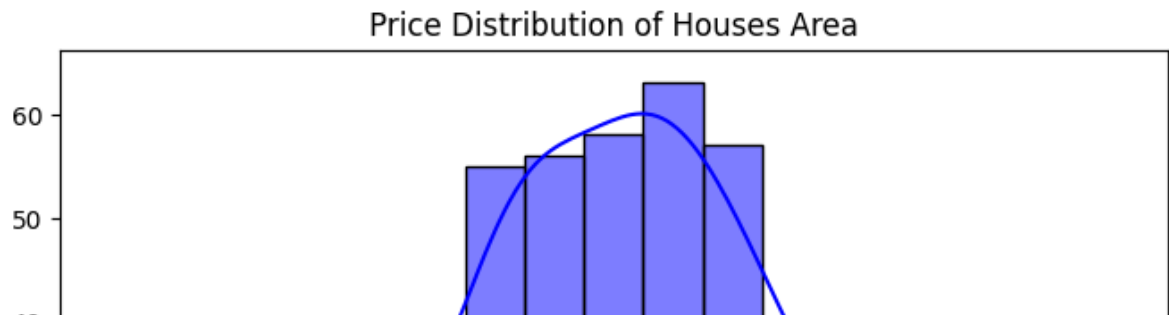


```

#SqFt_Living Distribution plot
plt.figure(figsize=(8, 5))
sns.histplot(df['SqFt_Living'], kde=True, color='blue')
plt.title('Price Distribution of Houses Area')
plt.xlabel('Square Feet')

```

```
plt.ylabel('Count')
plt.show()
```



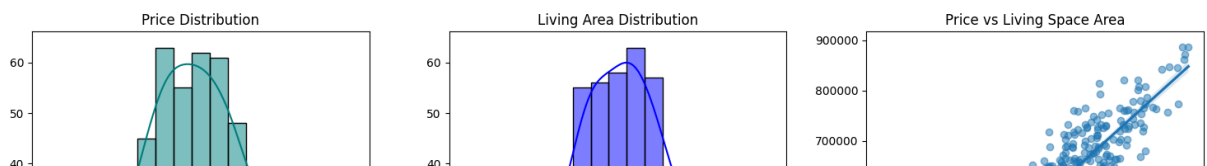
```
# Cell 9: 3 plots together
# Create figure with 1 row, 3 columns
plt.figure(figsize=(15, 5))

# Plot 1: Price Distribution
plt.subplot(1, 3, 1)
sns.histplot(df['Price'], kde=True, color='teal')
plt.title("Price Distribution")
plt.xlabel('Price')
plt.ylabel('Count')

# Plot 2: SqFt_Living Distribution
plt.subplot(1, 3, 2)
sns.histplot(df['SqFt_Living'], kde=True, color='blue')
plt.title('Living Area Distribution')
plt.xlabel('SqFt_Living')
plt.ylabel('Count')

# Plot 3: Price vs Living Space Regression plot
plt.subplot(1, 3, 3)
sns.regplot(data=df, x='SqFt_Living', y='Price', scatter_kws={'alpha': 0.5})
plt.title('Price vs Living Space Area')
plt.xlabel('SqFt_Living')
plt.ylabel('Price')

plt.tight_layout()
plt.show()
```



```
# Cell 10: ASSIGNMENT - Fill missing values

# Question 1: Fill SqFt_Living missing values with MEAN
sqft_mean = df['SqFt_Living'].mean()
print(f"\nMean of SqFt_Living: {sqft_mean:.2f}")

df['SqFt_Living'] = df['SqFt_Living'].fillna(sqft_mean)
```

```
# Verify
print(f"Missing after fill: {df['SqFt_Living'].isnull().sum()}")
```

```
Mean of SqFt_Living: 2123.34
Missing after fill: 0
```

```
# Question 2: Fill Bedrooms missing values with MEDIAN
bedrooms_median = df['Bedrooms'].median()
print(f"\nMedian of Bedrooms: {bedrooms_median:.0f}")

df['Bedrooms'] = df['Bedrooms'].fillna(bedrooms_median)

# Verify
print(f"Missing after fill: {df['Bedrooms'].isnull().sum()}")
```

```
Median of Bedrooms: 3
Missing after fill: 0
```

```
# Question 3: Display current missing values
print("=== BEFORE HANDLING MISSING VALUES ===")
print(f"Missing in SqFt_Living: {df['SqFt_Living'].isnull().sum()}")
print(f"Missing in Bedrooms: {df['Bedrooms'].isnull().sum()}")
print(f"Missing in other columns: \n{df.isnull().sum()}")
```

```
=== BEFORE HANDLING MISSING VALUES ===
Missing in SqFt_Living: 0
Missing in Bedrooms: 0
Missing in other columns:
House_ID                0
Neighborhood             0
House_Type              0
SqFt_Living             0
Bedrooms                0
Bathrooms               0
Year_Built              0
Distance_to_City_Center 0
Has_Pool                0
Condition_Score         0
Price                   0
Price_perSqFt           15
Status or Premium       0
dtype: int64
```

CODE EXPLANATION: "This notebook is an Exploratory Data Analysis of a housing market dataset with 500 rows and 11 columns. The goal is to understand the data structure, engineer new features, visualize distributions, and clean missing values before modeling."

Cell 1: Data Upload "I start by using `google.colab.files.upload()` to upload the CSV locally. This is necessary because Colab doesn't have access to my drive files by

default."

Cell 2: Imports and Data Loading "I import the standard data science libraries: `pandas` for dataframes, `numpy` for numerical operations, and `matplotlib`/`seaborn` for visualization. Then I load the CSV into a dataframe called `df` and use `df.head()` to confirm the first 5 rows loaded correctly."

Cell 3: Preview First 10 Rows "To get a slightly broader view than `head()`, I use `df.head(10)`. This helps me check for obvious data entry issues or column types early."

Cell 4: Data Structure with `.info()` "`.info()` shows me the column names, non-null counts, data types, and memory usage. From this I identified that `SqFt_Living` and `Bedrooms` have missing values, which we address later."

Cell 5: Descriptive Statistics "`.describe()` gives me mean, std, min, max, and quartiles for all numeric columns. This helps me understand price ranges, square footage spread, and detect outliers."

Cell 6: Feature Engineering "Here I create two new features:

1. `Price_perSqFt = Price / SqFt_Living` to normalize house price by size for fairer comparison.
2. `Status or Premium` using `np.where()`. Any house above the 75th percentile price is labeled 'premium', others 'standard'. This creates a binary target for potential classification later."

Cell 7 & 8: Univariate Distributions*

"I use `sns.histplot()` with KDE to visualize the distribution of `Price` and `SqFt_Living` separately. This shows if the data is skewed, normal, or has multiple modes before modeling."

Cell 9: Multivariate Visualization*

"I combine 3 plots in one figure with `plt.subplot(1,3,:)`.

1. Price distribution
2. `SqFt_Living` distribution
3. `sns.regplot` for Price vs `SqFt_Living` to check linear relationship. The regression line shows that larger living area generally correlates with higher price."

Cell 10: Handling Missing Values - Assignment Section*

"For the assignment:

1. *Question 1:* I imputed missing `SqFt_Living` values with the column mean using `fillna(sqft_mean)`. Mean is appropriate here because SqFt is continuous and

roughly symmetric.

2. Question 2: I imputed missing `Bedrooms` with the median using `fillna(bedrooms_median)`. Median is better for discrete/count data to avoid fractional bedrooms.
3. Question 3: I verified with `isnull().sum()` that both columns now have 0 missing values, and checked the rest of the dataframe."

Finally, the notebook moves from data ingestion -> inspection -> feature creation -> visualization -> data cleaning. After this, the dataset is ready for correlation analysis or a train/test split.